Lab 5: Model Selection Tools

NAME:

The kidney.csv file (in the Data folder on Canvas) contains a large data set from two kidney disease studies: Modification of Diet and Renal Disease (MDRD) and African American Study of Kidney Disease (AASK). There are various potential predictors in this dataset which measured patient characteristics related to their kidney disease, socio-economic measures of the patients, and other medical characteristics of the patients:

- **WEIGHT**: weight in kg
- **BMI**: body mass index
- **GFR**: glomerular filtration rate
- **UCRE**: urine creatinine
- **UUN**: urine urea nitrogen
- **UPHO**: urine phosphorus
- **SUN**: serum urea nitrogen
- **SCR**: serum creatinine
- **TCHOL**: total cholesterol
- **ALB**: albumin
- **HBA1C**: hemoglobin AIC
- **PHOS**: serum phosphorus
- **TRIG**: triglycerides
- **LDL**: low density lipoprotein (cholesterol)
- **HDL**: high density lipoprotein (cholesterol)
- **HB**: hemoglobin
- **MAP**: mean arterial pressure
- **UPRO**: urine protein
- **BSA**: body surface area
- **SODIUM**: sodium
- **GLUC**: glucose
- **BLACK**: black race (0/1)
- **HEIGHT**: height in cm
- **AGE**: age
- **FEMALE**: female (0/1)
- **CYS**: serum cystatin
- **DBP**: diastolic blood pressure
- **SBP**: systolic blood pressure

- **CRP**: C-reactive protein
- **DIAB**: diabetes
- **HBPSTATUS** high blood pressure (0/1)

The goal of this lab to determine which of these variables are associated with our outcome GFR (glomerular filtration rate) by building a predictive model. Here we note that GFR is a continuous measure of kidney function and lower GFR values indicate worse kidney function. We will demonstrate building a model using a) stepwise regression, b) ridge regression, and c) lasso regression. For this lab, we will just use the variables as they are given.

Load in the dataset and some useful packages.

```
library(dplyr)
library(ggplot2)
library(ggfortify)
library(GGally)
library(Hmisc)

kidney = read.csv("kidney.csv")
dim(kidney)
names(kidney)
```

First, we see that there are many incomplete rows of data. Looking further, we can see that 6 of the covariates (hbpstatus, UPHO, TRIG, LDL, HDL, and HBA1C) are each missing from more than 50% of the data. We choose to omit these predictors from our dataset since the proportion of missing data is so high. However, since there is not much overlap in the incompleteness between covariates, after removing those 6 variables and only retaining complete cases our dataset (N = 1547) is reduced to 56.27% of its original number of observations. (Note that we could have played around with other thresholds of missing data besides 50% and looked at the trade-off with the remaining dataset size.)

```
perc.missing = apply(apply(kidney, 2, is.na),2, mean)
kidney = kidney %>% select(-hbpstatus, -UPHO, -TRIG, -LDL, -HDL, -HBA1C)
kidney = na.omit(kidney)
dim(kidney)
attach(kidney)
```

Next, we also need to convert the following variables to factors:

```
kidney$BLACK = as.factor(BLACK)
kidney$FEMALE = as.factor(FEMALE)
kidney$Diabetes = as.factor(Diabetes)
```

In this lab, we will only perform some very basic EDA. Below we look at the correlations between the continuous variables and any relationships between GFR and the categorical variables. Three predictors that are highly correlated with GFR are cys, SUN, and SCR. Note that these variables themselves are highly linearly correlated.

```
ggcorr(kidney,label=TRUE)
ggplot(data=kidney) + geom_density(aes(x=GFR,fill=BLACK),alpha = 0.2)
ggplot(data=kidney) + geom_density(aes(x=GFR,fill=FEMALE),alpha = 0.2)
ggplot(data=kidney) + geom_density(aes(x=GFR,fill=Diabetes),alpha = 0.2)
```

Stepwise Variable Selection

In order to compare forward and backward stepwise selection, we will first split our dataset into a test and train dataset using the **sample** function. We choose to retain 75% of the data for training.

```
split = sample(1:nrow(kidney), size = floor(0.75*nrow(kidney)))
kidney.train = kidney[split,]
kidney.test = kidney[-split,]
```

Forward stepwise selection is a model section process that iteratively adds variables to the model based on some criteria (e.g. AIC). For this lab and for your homework, we recommend using the **step** function, which is part of the **stats** package. You can run forward stepwise selection as follows:

```
lm.null = lm(GFR~1,data=kidney.train)
lm.full = lm(GFR~.,data=kidney.train)
forward.lm = step(lm.null,scope=list(lower=lm.null,upper=lm.full),direction="forward")
```

Here, the first argument is the starting model. For forward selection, we start without any variables in the model (the 1 stands for the intercept). The second argument gives the scope of possible models, and the last argument gives the direction of adding variables (this can be forward, backward, or both). The printed output shows you the model after each step. You can also provide **glm** models to this function and can set the selection criteria to BIC (see ?step for more info).

The object returned by step can be treated like a linear model - we can view the coefficients, display diagnostic plots, and predict from the model.

```
summary(forward.lm)
autoplot(forward.lm)
forward.mse = mean((kidney.test$GFR - predict(forward.lm, kidney.test))^2)
```

How many variables are in the selected model? In what order were the variables added? If we are interested in the trade-off of selecting an even simpler model, we can also find the AIC for the intermediary models.

```
forward.lm$anova
ggplot()+geom_line(aes(x=1:12, y=forward.lm$anova$AIC))+ylab("AIC")+xlab("Number of Variables")
```

Do we see a big difference in AIC between 11 variables and 12 variables? How could we use cross-validation to get a better picture of the variance for choosing different numbers of variables?

Try this exercise out again but instead perform backwards selection. Compare the selected model to that of forward selection both in terms of the chosen variables, AIC, and test MSE. Which model would you choose between the two? In which scenarios do you think forward selection is more appropriate to use for variable selection? backward selection? Last, what might happen if we changed the selection criteria to BIC?

In this lab, we will skip best subsets selection. If you would like to learn it on your own, ISL Chapter 6.5 shows how to perform best subsets regression using the **leaps** package.

Ridge Regression and Lasso

We will use the **glmnet** function in the **glmnet** package to run ridge regression and lasso. In both cases, the function requires that the data be put into matrix format with a column corresponding to every variable. We can do this easily using the **model.matrix** function:

```
X=model.matrix(GFR~.,kidney)[,-1]
Y=kidney$GFR
```

This function will automatically convert categorical variables into dummy variable columns and will handle any non-linear terms. The [,-1] removes the intercept column, which is not necessary for the glmnet function.

Do you recall the optimization format for ridge regression? To run ridge regression we can run the following code:

```r
grid = 10^seq(10, -2, length = 100)
ridge.lm = glmnet(X,Y,alpha=0,lambda=grid)
```

First, we create a sequence of λ values to test stored in grid. Here, we chose 10^10 to 10^{-2}, which should cover the range from a model containing only the intercept to the OLS model. glmnet takes in the covariate matrix X, the response column Y, a value α indicating the optimization form ($\alpha = 0$ implies ridge regression, $\alpha = 1$ implies lasso), and the sequence lambda of λ values for which to run the ridge regression. By default, the function also standardizes the variables before running the optimization, but returns the coefficients in terms of the original values. We can always turn this off by adding **standardize=FALSE**. Why is standardization generally a good idea?

Try plotting the output. What does this plot show? What trends do you notice?

```r
plot(ridge.lm)
```

If we want to find the coefficients for a particular value of λ on our grid, we can use the **coef** function

```r
ridge.lm$lambda[20]
coef(ridge.lm)[,20]
ridge.lm$lambda[50]
coef(ridge.lm)[,50]
```

What do you notice about the coefficients for the first value of λ vs the second?

Of course, given just this output it is hard to determine which value of λ to pick. Below we use 10-fold cross validation to find the "best" value of λ by estimating the test MSE for each value of λ on the grid.

```r
# 10-fold cross-validation for ridge
folds = sample(1:10, nrow(kidney), replace = TRUE)
cv.mse = matrix(NA, nrow = 10, ncol = 100)

for (i in 1:10){
  ridge.lm=glmnet(X[folds!=i,],Y[folds!=i],alpha=0,lambda=grid)
  for (j in 1:100){
    cv.mse[i,j] = mean((predict(ridge.lm,s=grid[j],newx=X[folds==i,])-Y[folds==i])^2)
  }
}
avg.cv.mse = apply(cv.mse,2,mean)
sd.cv.mse = apply(cv.mse,2,sd)

ridge.lambda.min = grid[which.min(avg.cv.mse)]
ggplot()+geom_line(aes(x=log(ridge.lm$lambda),y=avg.cv.mse)) +
  geom_errorbar(aes(x=log(ridge.lm$lambda),ymin=avg.cv.mse-sd.cv.mse,
                    ymax=avg.cv.mse+sd.cv.mse),col="gray") +
  xlab("Log(lambda)")+ylab("Avg MSE")+
  geom_vline(aes(xintercept=ridge.lambda.min),col="blue")
```

Walk through the code to make sure you understand what each piece is doing. What is the value of λ chosen by cross-validation? Find the coefficients corresponding to this value of λ for the model fitted on the *whole* dataset. Which variables seem to be the most important?

Let's walk through these steps again but for lasso. Recall that do so we just alpha = 1.

```r
lasso.lm = glmnet(X,Y,alpha=1,lambda=grid)
plot(lasso.lm)
```

What do you notice about the coefficient plot for lasso vs ridge regression? Why do we observe this difference?

Suppose that after running the function above we actually want to find the coefficients and predicted values for a value of λ not on our grid. We can do so using the **predict** function.

```
predict(lasso.lm,type="coefficients",s=15)[1:20,]
predict(lasso.lm,s=15,X)
```

The $s = 15$ specifies to set $\lambda = 15$.

Try running 10-fold cross-validation to choose the best value of λ for lasso. However, instead of modifying the code above, use the built-in function **cv.glmnet**.

```
cv.lasso=cv.glmnet(X,Y,alpha=1,nfolds=10,type.measure="mse")
names(cv.lasso)
cv.lasso$cvm
plot(cv.lasso)
lasso.lambda.min=cv.lasso$lambda.min

# Model with best lambda fit on whole dataset
predict(lasso.lm,type="coefficients",s=lasso.lambda.min)[1:20,]
```

This did a lot of the work for us! Note that **cvm** gives the estimated MSE for each value of λ and **cvsd** gives the standard deviation. What do you notice about model chosen by CV for lasso vs that of ridge regression? How would you compare both of these methods to stepwise selection (advantages, disadvantages, output, etc.)?